PROPUESTA TÉCNICA Y COMERCIAL · VIVA AEROBUS

Integración con la Plataforma Única de Identidad (PUI) en AWS, entregada por iterables de valor

Cinco entregas que se pueden liberar a Producción por sí solas: primero la PUI registra sus casos, luego encontramos y notificamos coincidencias, después buscamos hacia atrás, vigilamos y aseguramos que nada se pierda. Once semanas de construcción, con seguimiento semanal, infraestructura como código y pruebas dentro de cada iterable.

Así entendemos la interacción: la PUI, nuestra solución y la API interna de Viva



1. **Activa el caso.** La PUI se autentica en nuestro `/login` y llama a `/activar-reporte`; guardamos el caso con la CURP cifrada y respondemos de inmediato.
2. **Buscamos la CURP.** Fase 1 sin fechas, fase 2 con ventana de hasta 12 años y fase 3 periódica mientras el caso siga activo.
3. **La API interna responde** con la persona y sus eventos (reservación, check-in, abordaje, empleo).
4. **Reportamos.** Enviamos `/notificar-coincidencia` con los datos que autorice Legal y cerramos el histórico con `/busqueda-finalizada`.
5. **Desactiva.** La PUI llama a `/desactivar-reporte` y detenemos búsquedas y envíos del caso.

Las cajas punteadas son precondiciones. Con la entrega por iterables ya no hace falta tenerlas todas el primer día: cada iterable requiere lo suyo, listo antes de arrancarlo (lámina 8). La especificación v1.3 de la API interna es una buena base, pero le falta paginación por cursor y un token de sincronización para las fases 2 y 3 (lámina 9).

Cinco iterables de valor: cada uno se puede liberar a Producción por sí solo

ITERABLE 1 · LIBERABLE EN LA SEMANA 6

1 La PUI registra y da de baja sus casos

Se autentica, activa, prueba y desactiva reportes por CURP. Cada caso queda cifrado, sin duplicados y auditado desde la primera llamada.

ITERABLE 2 · LIBERABLE EN LA SEMANA 8

2 Encontramos lo más reciente y lo notificamos

Fase 1: buscamos la CURP en Reservaciones y Recursos Humanos y enviamos a la PUI la coincidencia, solo con los datos que aprueba Legal y con reintentos.

ITERABLE 3 · LIBERABLE EN LA SEMANA 9

3 Buscamos hasta 12 años atrás y cerramos el histórico

Fase 2: cada vuelo, check-in, abordaje o empleo desde la fecha de desaparición se notifica por separado y se cierra con `/busqueda-finalizada`.

ITERABLE 4 · LIBERABLE EN LA SEMANA 10

4 Seguimos vigilando mientras el caso esté activo

Fase 3: con la frecuencia que defina Viva, revisamos solo lo nuevo o modificado y lo notificamos, hasta que la PUI desactiva el caso.

ITERABLE 5 · LIBERABLE EN LA SEMANA 11

5 Nada se pierde: conciliación y reproceso

Comparamos con `/reportes` de la PUI, reprocesamos lo fallido con control y cerramos con la regresión y la aceptación final.

EN TODOS LOS ITERABLES

Lo que exige Producción

Pruebas y regresión, corrección de los hallazgos de la revisión de seguridad de Viva, bitácora de auditoría, trazabilidad, métricas y alarmas, IaC con reversa, runbook y aceptación de Viva (lámina 6).

Ningún iterable deja nada para el final: cada uno sale probado, seguro, auditado, monitoreado y desplegable con el pipeline, y Viva decide cuándo lo lleva a Producción. Recomendamos conectar con la PUI cuando las tres fases estén en Producción (iterable 4), porque el manual exige las tres; el iterable 5 refuerza la operación.

Iterables 1 y 2: recibir el caso y reportar la primera coincidencia

ITERABLE 1 · SEMANAS 1 A 6 · LIBERABLE AL CIERRE DE LA SEMANA 6

La PUI registra y da de baja sus casos

Viva cumple con los endpoints de entrada del manual, con todo lo que Producción exige.

Se construye	`/login`, `/activar-reporte`, `/activar-reporte-prueba` y `/desactivar-reporte`; validación de CURP, idempotencia, firmado del JSON y OpenAPI.
Infraestructura que nace	Terraform base por ambiente, KMS, IAM, Cognito, API Gateway, WAF, dominio y TLS, tabla de casos; pipeline con promoción, reversa y smoke tests.
Se prueba	Funcionales de entrada, scopes y CURP inválidas; contrato automatizado; carga con Celerabit; conectividad en el sandbox. Viva revisa su seguridad y corregimos los hallazgos.
Auditoría y operación	Bitácora inmutable con consultas de auditoría, `correlationId`, métricas y alarmas de entrada, CloudTrail y runbook.
Requiere antes de arrancar	AWS, repositorio y Terraform; fecha de la revisión de seguridad de Viva; dominio y certificados; retención de la bitácora y acceso al sandbox de la PUI.

Deja lista la base de Producción que heredan los demás: por eso es el iterable más grande (50 % del alcance).

ITERABLE 2 · SEMANAS 5 A 8 · LIBERABLE AL CIERRE DE LA SEMANA 8

Encontramos lo más reciente y lo notificamos

La PUI empieza a recibir lo que Viva sabe de la persona: el primer valor real para la búsqueda.

Se construye	Cliente de la API interna, estados del caso, fase 1, cliente y token de la PUI, notificador con reintentos y mapeo al contrato de la PUI.
Infraestructura que nace	Red privada, NAT con IP fija, colas SQS de búsqueda y notificación con DLQ, y secretos de la PUI y de la API interna.
Se prueba	Conexión a cada fuente, fase 1, notificación, reintentos y DLQ con simuladores; datos enviados frente a la matriz de Legal; regresión del iterable 1.
Auditoría y operación	Cada consulta a la API interna y cada envío a la PUI quedan en la bitácora; métricas de latencia y errores, alarmas de colas y DLQ, runbook.
Requiere antes de arrancar	El contrato v1.3 congelado. Antes de la semana 7, cuando empieza la búsqueda real: conexión a la API interna con CURP de prueba y matriz de datos aprobada por Legal.

El cliente de la API interna se construye desde la semana 5 contra un simulador del contrato v1.3, mientras se cierra el iterable 1.

Iterables 3 y 4: buscar hacia atrás y seguir vigilando

ITERABLE 3 · SEMANAS 8 Y 9 · LIBERABLE AL CIERRE DE LA SEMANA 9

Buscamos hasta 12 años atrás y cerramos el histórico

La PUI recibe el rastro histórico completo de la persona y sabe cuándo terminó la búsqueda.

Se construye	Worker de fase 2 por páginas, desde la fecha de desaparición con tope de 12 años; una notificación por evento y cierre con `/busqueda-finalizada`, haya o no coincidencias.
Infraestructura que se agrega	Cola de fase 2 por páginas y registro de cursores. Se enciende por parámetro, sin tocar lo ya liberado.
Se prueba	Fase 2 por páginas, caso sin fecha de desaparición, reintento de una sola página y cierre; regresión de los iterables 1 y 2.
Auditoría y operación	Cada página consultada y cada notificación de fase 2 quedan en la bitácora; duración de la fase en el tablero.
Requiere antes de arrancar	Decisión sobre la paginación por cursor, tomada en la semana 4 (lámina 9), y cursor disponible en desarrollo.

ITERABLE 4 · SEMANAS 8 A 10 · LIBERABLE AL CIERRE DE LA SEMANA 10

Seguimos vigilando mientras el caso esté activo

Si la persona aparece en una reserva nueva, la PUI se entera en el siguiente ciclo, sin esperar a otra búsqueda completa.

Se construye	Planificador de casos activos y worker de fase 3 con token de sincronización o, si la API interna no lo ofrece, por fecha de modificación.
Infraestructura que se agrega	EventBridge Scheduler, Lambda planificadora, cola de fase 3 y almacenamiento del token. También se enciende por parámetro.
Se prueba	Fase 3 con token y con su alternativa, token vencido y desactivación; pruebas funcionales en el sandbox de la PUI; regresión de los iterables 1 a 3.
Auditoría y operación	Cada ciclo de fase 3 queda en la bitácora; alarma si la fase 3 deja de correr; runbook de las fases 2 y 3.
Requiere antes de arrancar	Token de sincronización en desarrollo (o usar fecha de modificación), frecuencia de la fase 3 y volumen esperado.

Los iterables 3 y 4 se construyen en paralelo, porque ambos reutilizan el cliente de la API interna y el notificador del iterable 2. Dependen del cambio de la lámina 9: si Viva no lo implementa, se usa la alternativa de consultas por año y por fecha de modificación, con más carga sobre la API interna.

Iterable 5 y lo que cumple cada iterable antes de ir a Producción

ITERABLE 5 · SEMANAS 10 Y 11 · LIBERABLE AL CIERRE DE LA SEMANA 11

Nada se pierde: conciliación y reproceso

La red de seguridad de la operación: si algo falla, se detecta, se corrige y queda evidencia.

Se construye	Conciliación periódica con `/reportes` de la PUI y reproceso controlado desde la DLQ.
Infraestructura que se agrega	Lambdas conciliadora y de reproceso, con su programación en EventBridge.
Se prueba	Resiliencia ante caídas de la PUI y de la API interna, reproceso y conciliación; regresión final de extremo a extremo y aceptación final.
Auditoría y operación	Cada corrección y reproceso queda en la bitácora; corrección de los hallazgos de la revisión final de Viva, documentación final y transferencia.
Requiere antes de arrancar	Quién recibe y atiende las alertas de Producción.

DEFINICIÓN DE LIBERABLE · SE CUMPLE EN LOS 5 ITERABLES

Un iterable no se libera si le falta algo de esto

Pruebas	Unitarias, funcionales, de contrato y regresión de todo lo ya liberado.
Seguridad	Viva revisa su seguridad con sus herramientas y corregimos los hallazgos antes de liberarlo.
Auditoría	Todas sus interacciones quedan en la bitácora inmutable.
Trazabilidad	`correlationId` de punta a punta; la CURP nunca aparece en logs, URLs ni errores.
Operación	Métricas, alarmas y su sección del runbook.
Despliegue	IaC y pipeline con promoción, reversa y smoke tests.
Aceptación	Viva lo aprueba antes de liberarlo, y acompañamos la liberación.

Así se ve en Producción: si Viva libera cada iterable al aceptarlo, el 1 está en Producción desde la semana 7, el 2 desde la 9 y el 3 desde la 10. Con los iterables 4 y 5 en la semana 12, Viva puede conectar con la PUI con las tres fases. Las semanas 12 y 13 son de soporte (lámina 19).

¿Es viable en 11 semanas? Sí, con condiciones. Y esto es lo que no cabe

1 Lo que sí entregamos en 11 semanas

- Los 5 iterables construidos, probados, auditados y aceptados, cada uno liberable a Producción.
- Los iterables 1 a 3 en Producción dentro de las 11 semanas, si Viva los libera al aceptarlos (semanas 7, 9 y 10).
- El paquete que pide la PUI, acumulado iterable a iterable: bitácora y evidencias funcionales; los reportes de seguridad salen de la revisión de Viva.
- Pruebas en el sandbox de la PUI, si se habilita a tiempo.
- Runbook, documentación y transferencia de conocimiento.

2 Condiciones para que sea viable

- **Revisión de seguridad de Viva en cada liberación**, con hallazgos entregados en 3 días hábiles.
- **Lo que requiere cada iterable, listo antes de que arranque** (lámina 8).
- **API interna real en desarrollo** antes de la semana 7.
- **Paginación decidida en la semana 4**; cursor y token de sincronización en desarrollo antes de la semana 8.
- **Usuarios de negocio desde el inicio**, alineados con lo que recibirán; aceptan cada iterable en 2 días hábiles y Viva lo libera con el pipeline.

3 Lo que no es viable

- **Operar con la PUI con las tres fases dentro de las 11 semanas:** los iterables 4 y 5 se aceptan en las semanas 10 y 11 y se liberan, como pronto, en la 12; el soporte de 2 semanas es en la 12 y la 13.
- **Mantener la fecha si algo de lo que requiere un iterable no está listo antes de arrancarlo:** ese iterable se corre lo mismo, y con él la fecha final.
- **Liberar con la PUI sin acceso al sandbox:** solo se valida con simuladores.

La regla de holgura: las pruebas, la seguridad y la auditoría corren dentro de cada iterable, así que cada cierre confirma y no descubre. El tramo más exigente son las semanas 1 a 6, porque el iterable 1 deja lista toda la base de Producción. Si la holgura se agota antes de la semana 9, o si la revisión de seguridad de Viva exige más de un ciclo de correcciones, el plan pasa a 12 semanas y lo avisamos en el seguimiento semanal.

Lo que requiere cada iterable antes de arrancar

Viva debe tener listo lo que requiere cada iterable **antes de que arranque**. Si algo falta, **ese iterable no arranca y la fecha final se corre lo mismo**. Lo revisamos cada semana con los responsables de Viva.

ANTES DE LA SEMANA 1

Para arrancar el proyecto

Requiere

- Cuenta de AWS de desarrollo con permisos, red, salida aprobada y MFA.
- Repositorio y CI/CD en GitLab (por confirmar), OIDC hacia AWS y Terraform con estado.
- Patrocinador de Viva y un responsable por cada precondición.
- Proyecto en Jira.
- Usuarios de negocio asignados desde el inicio: acuerdan qué recibirán en cada iterable y lo reciben en cuanto está listo.

ITERABLE 1 · ARRANCA EN LA SEMANA 1 · LIBERABLE AL CIERRE DE LA 6

La PUI registra y da de baja sus casos

Requiere antes de arrancar

- Responsable y fechas de la revisión de seguridad de Viva, y si la PUI acepta sus reportes.
- Dominio, DNS, certificados e IP de origen de la PUI.
- Retención y acceso de la bitácora (Legal).
- Registro de Viva en la PUI y acceso al sandbox.

Para liberarlo

- QA y Producción listos con paridad certificada, ventana de Celerabit y hallazgos de seguridad de Viva entregados.

ITERABLE 2 · ARRANCA EN LA SEMANA 5 · LIBERABLE AL CIERRE DE LA 8

Encontramos lo más reciente y lo notificamos

Requiere antes de arrancar

- Contrato v1.3 de la API interna congelado, para construir contra un simulador.

Requiere antes de la semana 7, cuando empieza la búsqueda real

- Conexión a Reservaciones y Recursos Humanos en desarrollo, con CURP de prueba.
- Matriz de datos aprobada por Legal y regla para varias coincidencias.

ITERABLE 3 · ARRANCA EN LA SEMANA 8 · LIBERABLE AL CIERRE DE LA 9

Histórico de hasta 12 años

Requiere antes de arrancar

- Decisión sobre la paginación por cursor, tomada en la semana 4 (lámina 9).
- Cursor disponible en desarrollo.

ITERABLE 4 · ARRANCA EN LA SEMANA 8 · LIBERABLE AL CIERRE DE LA 10

Vigilancia continua

Requiere antes de arrancar

- Token de sincronización en desarrollo, o la decisión de usar fecha de modificación.
- Frecuencia de la fase 3 y volumen esperado de casos.

Para liberarlo

- Ventana de Celerabit del flujo completo y ambiente que se certifica con el sandbox.

ITERABLE 5 · ARRANCA EN LA SEMANA 10 · LIBERABLE AL CIERRE DE LA 11

Conciliación y reproceso

Requiere antes de arrancar

- Quién recibe y atiende las alertas de Producción.

Solicitud de cambio a la API interna de Viva: paginación por cursor

1 Por qué

- Una persona puede acumular cientos o miles de eventos en 12 años, y la v1.3 los devuelve en una sola respuesta.
- Una respuesta así pone en riesgo los tiempos y la memoria de la API interna y de nuestras Lambdas.
- Filtrar por fecha del evento no detecta en la fase 3 los cambios sobre eventos antiguos.

2 Cambio al contrato `POST /v1/person-records/search`

```
// Fases 1 y 2: recorrer por páginas
{ "curp": "XEXX010101HNEXXXA0",
  "fromDate": "2014-10-22", "toDate": "2026-10-22",
  "pageSize": 100, "cursor": null }

// Respuesta
{ "results": [ ... hasta 100 eventos ... ],
  "page": { "nextCursor": "eyJ1IjoiMjAxOS...", "hasMore": true },
  "syncToken": null }

// Fase 3: solo lo nuevo o modificado
{ "curp": "XEXX010101HNEXXXA0", "syncToken": "c3luYzoyMDI2..." }
```

3 Reglas que debe cumplir

- **Orden estable** por `updatedAt` y `eventId`, igual en todas las páginas.
- **Cursor por posición del último elemento** (keyset), no por número de página: así un evento nuevo no corre la lista.
- **Cursor opaco**, con vencimiento documentado.
- **`pageSize` máximo acordado**, por ejemplo 100 eventos.
- **`syncToken` en la última página.** Con él se devuelve solo lo creado, modificado o cancelado desde entonces.
- **Error 410 si el token venció**; en ese caso hacemos una sincronización completa.

4 Cuándo lo necesitamos

- **Semana 4:** la decisión de Viva, para diseñar las fases 2 y 3.
- **Antes de la semana 8:** el cursor en desarrollo, para arrancar el iterable 3.
- **Antes de la semana 8:** el token de sincronización en desarrollo, para arrancar el iterable 4.

Si no se implementa: la fase 2 se parte en consultas de un año y la fase 3 filtra por fecha de modificación (`updatedSince`). Funciona y no mueve el plan, pero con más carga sobre la API interna y sin garantía cuando un año concentra muchos eventos. Los iterables 1 y 2 no dependen de este cambio.

Cómo funciona (1 de 3): recibir el caso y buscar lo más reciente

Ejemplo: la PUI reporta la CURP `XEXX010101HNEXXXA0`. La solución funciona como intermediario: recibe el caso, busca en la API interna de Viva y le reporta a la PUI lo que encuentra.

ENTRADA · ITERABLE 1

La PUI entrega el caso

1. La PUI se autentica en nuestro `/login` y recibe un JWT.
2. Llama a `/activar-reporte` con la CURP; WAF y API Gateway validan tráfico y token antes de ejecutar código.
3. Una Lambda valida la CURP, evita duplicados, guarda el caso cifrado en DynamoDB y, desde el iterable 2, deja un mensaje en la cola SQS.
4. La PUI recibe «recibido» en segundos; la búsqueda espera en la cola y se reintenta si la API interna falla.

FASE 1 · ITERABLE 2

Lo más reciente

5. El worker de fase 1 toma el mensaje de la cola.
6. Consulta la API interna sin fechas; esta busca en Reservaciones (Navitaire) y en Recursos Humanos.
7. La API responde, por ejemplo, con una reserva próxima y los datos de contacto.
8. Si hay datos, deja un único aviso en la cola de notificación; si no hay, la fase 1 no notifica.

NOTIFICACIÓN · ITERABLE 2

Reportar a la PUI por servicio web

9. La Lambda notificadora toma el aviso de nuestra cola interna; la PUI no ve esa cola.
10. Obtiene el JWT de la PUI (vigencia de una hora) con credenciales de Secrets Manager.
11. Llama por HTTPS a `/notificar-coincidencia` con `fase_busqueda` y solo los campos aprobados por Legal.
12. Si la PUI no responde, el aviso vuelve a la cola y se reintenta; tras varios intentos pasa a la DLQ.

Cómo funciona (2 de 3): buscar hacia atrás y seguir vigilando

FASE 2 ITERABLE 3

Histórico de hasta 12 años

13. Al terminar la fase 1, el worker consulta la API interna desde la fecha de desaparición que envía la PUI, con tope de 12 años. Sin esa fecha, la fase se omite (por confirmar si igual se envía el cierre).
14. La API interna entrega los eventos por páginas. Cada página es un mensaje en la cola y, si una falla, se reintenta solo esa página.
15. Cada evento encontrado (un vuelo, un check-in, un empleo) se notifica por separado con el paso de notificación.
16. Cuando la última página de cada fuente llega sin más resultados, se envía `/busqueda-finalizada`, aunque no haya coincidencias.

FASE 3 ITERABLE 4

Vigilancia mientras el caso esté activo

17. EventBridge funciona como despertador, con la frecuencia que defina Viva.
18. Una Lambda planificadora lee en DynamoDB los casos activos y los pone en la cola de fase 3.
19. El worker envía el token de sincronización guardado y la API interna devuelve solo lo creado, modificado o cancelado desde la última consulta.
20. Si hay algo nuevo, por ejemplo una reserva creada ayer, se notifica y se guarda el token nuevo.

Esto requiere el cambio de la lámina 9 en la API interna: paginación por cursor y token de sincronización. Sin él, la fase 2 se parte en consultas de un año y la fase 3 filtra por fecha de modificación.

Cómo funciona (3 de 3): cerrar el caso y operar con seguridad

DESACTIVACIÓN · ITERABLE 1

La PUI cierra el caso

21. Cuando el caso se resuelve, la PUI llama a `/desactivar-reporte`.
22. Una Lambda marca el caso como inactivo. La fase 3 deja de incluirlo y no se envían más notificaciones.

CONCILIACIÓN Y AUDITORÍA · AUDITORÍA DESDE EL 1

Red de seguridad y evidencia

23. Cada cierto tiempo, una Lambda compara `/reportes` de la PUI con los casos guardados.
24. Si algo no cuadra, se corrige.
25. Cada activación, consulta y envío queda en una bitácora que nadie puede modificar ni borrar.
26. Para auditar se consulta qué se envió, cuándo y qué respondió la PUI; ver los datos exige un permiso especial.

TRANSVERSAL · ITERABLES 1 A 5

Lo que rodea todo

- **Red privada:** las Lambdas no son accesibles desde Internet; API Gateway es la única puerta de entrada.
- **Cifrado:** KMS protege la CURP, que nunca se escribe en los logs.
- **Monitoreo:** CloudWatch muestra métricas y alerta, por ejemplo, si se acumulan fallidos o la fase 3 deja de correr.
- **IaC y pipeline:** todo está escrito como código y crece con cada iterable, para que Viva cree lo mismo en QA y Producción.

En una frase: la PUI entrega una CURP; la solución la guarda y la busca en la API interna de Viva, primero lo reciente, luego 12 años atrás y después de forma periódica. Cada hallazgo se reporta a la PUI con reintentos, hasta que la PUI cierra el caso.

Lo que entregamos: cada iterable suma su parte de cada bloque

1 Arquitectura y diseño

- Arquitectura lógica, física y de despliegue.
- Diagramas de secuencia de las tres fases.
- Modelo de datos y estados del caso.

2 Contratos

- OpenAPI 3.x y JSON Schemas de `/login`, `/activar-reporte`, `/activar-reporte-prueba` y `/desactivar-reporte`.
- Contrato del cliente de la API interna y del cliente de la PUI.

3 Código e infraestructura

- Código Node.js de las Lambdas y la librería común.
- IaC en Terraform por módulos de capacidad, parametrizada por ambiente, con dominio propio y TLS.
- Pipelines de build, IaC y promoción, con reversa y smoke tests.

4 Pruebas y evidencias

- Pruebas unitarias de cada historia, incluidas aunque la práctica habitual de Viva no las tiene.
- Evidencias por iterable de pruebas de contrato, integración, resiliencia y extremo a extremo (lámina 14).
- Pruebas de aceptación con el negocio de Viva y matriz de trazabilidad.
- Pruebas de desempeño, sujetas al alcance y capacidad acordados con Celerabit (lámina 15).
- Evidencias de la certificación en el sandbox de la PUI, si se habilita.

5 Seguridad

- Controles de seguridad desde el diseño: autenticación, scopes, WAF, cifrado y red privada.
- Corrección de los hallazgos de la revisión de seguridad que ejecuta Viva, en cada liberación (lámina 16).
- Firmado del JSON de respuesta que exige la PUI.
- Bitácora de auditoría inmutable y evidencia de que la CURP no aparece en logs.

6 Operación

- Dashboard y alarmas de los 9 indicadores de la sección 13 del diseño, con trazabilidad por `correlationId`.
- Runbook, plan de despliegue y reversa.
- Estimación del costo de operación en AWS.
- Una liberación por iterable, reporte semanal de avance y transferencia de conocimiento.

Fuera de alcance: construir la API interna de Viva o habilitar el acceso a Navitaire; fotos y huellas; ejecutar pruebas de seguridad (SAST, SCA, DAST, de las APIs o de penetración), que hace Viva; y ejecutar pruebas en Producción. Después de las 2 semanas de soporte incluidas (lámina 19), la continuidad se cotiza aparte.

Calidad en cada iterable, de la mano del negocio

La calidad acompaña el proyecto desde la semana 1: al inicio se definen con el negocio los criterios, los datos y los requerimientos de prueba, y cada entrega se prueba en cuanto se despliega en el ambiente de desarrollo. Las pruebas se ejecutan dentro de cada iterable, a medida que se construye, y ninguno se da por terminado sin sus evidencias. Al final solo quedan la regresión y la aceptación, que confirman lo ya probado.

1 · ANTES DE CODIFICAR

Criterios que Viva aprueba

- Criterios de aceptación por iterable, acordados desde el inicio con los usuarios de negocio, en formato Dado / Cuando / Entonces.
- Reglas del negocio verificables: matriz de datos, fases e idempotencia.
- Trazabilidad de requisito a evidencia.

2 · ITERABLE A ITERABLE

Terminado quiere decir probado

- Cada iterable cierra con pruebas funcionales, de contrato y regresión de lo ya liberado.
- Contrato, validación de CURP, idempotencia, scopes por ruta y manejo de errores.
- Las pruebas unitarias se escriben al construir cada historia.

3 · INTEGRACIÓN VALOR AGREGADO

Automatización acotada de APIs

- Base automatizada de contrato y smoke/regresión para los 4 endpoints expuestos, que corre en el pipeline desde el iterable 1.
- Simuladores de la PUI y de la API interna desde el iterable 2, para no esperar a los sistemas reales.
- Las pruebas exploratorias conservan ejecución manual.

4 · PRUEBAS DE ACEPTACIÓN

Con el negocio de Viva

- Viva acepta cada iterable antes de liberarlo; la aceptación final es en la semana 11.
- Tech and Solve prepara, ejecuta y documenta las evidencias.
- Certificación de la integración en el sandbox de la PUI, si se habilita.

Desarrollo
Pruebas unitarias de cada historia.

Calidad
Funcionales, contrato, integración, resiliencia, extremo a extremo y aceptación.

Celerabit
Desempeño, sin costo adicional (lámina 15).

Viva
Revisión de seguridad; corregimos sus hallazgos (lámina 16).

Pruebas de desempeño con Celerabit

El servicio de desempeño que Viva ya tiene.

QUÉ PROBAREMOS DE LA INTEGRACIÓN

- Los endpoints de entrada (token, activar y desactivar) bajo carga: latencia p90, p95 y p99, rendimiento y errores.
- El flujo asíncrono con simuladores: colas, workers y notificador frente a los límites de Navitaire y de la PUI.

DISPARO DESDE EL PIPELINE

- La suite se puede disparar desde el pipeline de CI/CD, según las herramientas, permisos y definición acordada con Viva.
- Umbrales de latencia y error para detectar degradaciones entre versiones.

CUÁNDO

- Desde el inicio: definimos con Viva los requerimientos de desempeño (volúmenes, umbrales y escenarios).
- Se ejecuta cuando la solución esté desplegada y estable: la entrada antes de liberar el iterable 1 y el flujo completo antes de los iterables 4 y 5.

$0 **adicionales en este proyecto:** las pruebas de desempeño se ejecutan dentro del plan de Celerabit que Viva ya tiene contratado.

Seguridad: Viva revisa y nosotros corregimos en cada liberación

1 Lo que construimos seguro desde el diseño

- **Autenticación y autorización:** /login con Cognito y un scope por ruta.
- **Entrada protegida:** WAF, límites de tráfico y validación de CURP y contrato.
- **Datos:** cifrado con KMS; la CURP no aparece en logs, URLs ni errores; solo salen los campos aprobados por Legal.
- **Red privada** y firmado del JSON de respuesta que exige la PUI.

2 Lo que hace Viva

- Ejecuta la revisión de seguridad con sus herramientas y su equipo: SAST, SCA, DAST y pruebas de las APIs.
- Entrega los hallazgos clasificados por severidad, con evidencia, en 3 días hábiles.
- Valida las correcciones y genera los reportes que pide la PUI.

3 Lo que hacemos nosotros y cuándo

- **Corregimos los hallazgos** sobre la solución en el iterable en que se reportan, y corremos la regresión de lo ya liberado después de cada corrección.
- **Semana 5 · iterable 1:** Viva revisa la entrada; corregimos en la 6.
- **Semana 8 · iterable 2:** revisión de la búsqueda y la notificación, y corrección.
- **Semana 10:** revisión final; corregimos en la 11.

Incluye un ciclo de corrección por cada revisión. El manual de la PUI exige que la URL base y los endpoints queden libres de vulnerabilidades críticas, altas, medias y bajas; Viva lo verifica y nosotros corregimos lo que corresponda a la solución. Fuera de alcance: ejecutar pruebas de seguridad o de penetración, emitir informes de seguridad, hallazgos sobre la PUI, Navitaire u otros sistemas de terceros, y ciclos de corrección adicionales, que se estiman aparte.

Dudas a resolver antes de iniciar

1 ¿Cómo y cuándo revisa Viva la seguridad?

- ¿Quién ejecuta la revisión (SAST, SCA, DAST y APIs) y con qué herramientas?
- ¿En qué ambiente y en qué momento de cada iterable? ¿Entrega los hallazgos en 3 días hábiles?
- ¿Tendremos acceso a los resultados (por ejemplo, el SonarQube de Viva) y la PUI acepta esos reportes?

Se necesita antes de arrancar el iterable 1: define cuándo llegan los hallazgos y cuándo se corrigen.

2 ¿Viva liberará cada iterable al aceptarlo?

- **Sí (recomendado):** la primera liberación es en la semana 7, las diferencias de ambiente aparecen temprano y cada liberación es pequeña y con reversa.
- **Solo al final:** funciona, pero todo el riesgo se concentra en la semana 12.
- ¿Y cuándo se conecta con la PUI: con las tres fases en Producción (recomendado) o antes, si la PUI lo permite?

Define cuándo aparecen los riesgos de ambiente y cuándo opera Viva con la PUI.

3 ¿Qué herramientas de IaC y CI/CD usa Viva?

- Por defecto proponemos Terraform. ¿Viva lo tiene habilitado o exige otra herramienta (AWS CDK, CloudFormation/SAM)?
- ¿Dónde se guarda el estado y hay módulos o plantillas corporativas obligatorias?
- ¿El repositorio y el pipeline están en GitLab? Si es así, ¿GitLab CI puede desplegar con OIDC en cada cuenta de AWS?

Se necesita el día 1: es lo primero que se construye en el iterable 1.

Otras dudas abiertas.
Con la PUI: si se puede certificar por partes o solo la solución completa; si hay que firmar el JSON de respuesta y qué significa «tokens no reutilizables»; si se envía `/busqueda–finalizada` cuando no hay fecha de desaparición; si tendremos cuenta propia de sandbox; y si exige pruebas de penetración de un tercero (en ese caso, Viva las contrata con su proveedor).
Con Viva: si implementa la paginación de la lámina 9; quién recibe y atiende las alertas (y si van también a Datadog); y si el «soporte operativo» del diseño requiere continuidad después de las 2 semanas de soporte incluidas (lámina 19).

Términos y condiciones

1 Alcance

- Incluye los 5 iterables (láminas 3 a 6) y lo de la lámina 13, con acompañamiento a cada liberación y 2 semanas de soporte en Producción (lámina 19).
- No incluye construir la API interna ni el acceso a Navitaire, fotos y huellas, ni ejecutar pruebas de seguridad o de penetración: las hace Viva y corregimos sus hallazgos, un ciclo por revisión.
- La continuidad del soporte después de esas 2 semanas se cotiza aparte.

2 Ambientes

- Tech and Solve solo accede a desarrollo.
- Viva despliega QA y Producción con el pipeline entregado y puede liberar cada iterable al aceptarlo, desde la semana 7.
- Antes de la primera liberación, Viva certifica por escrito que QA y Producción son iguales a desarrollo salvo parámetros y secretos.
- Los ajustes por diferencias de ambiente se estiman como actividad nueva.

3 Dudas abiertas

- Las dudas de la lámina 17 se resuelven en las fechas que ahí se indican.
- El firmado del JSON ya está incluido; otra respuesta que cambie el alcance se estima como actividad adicional.
- Lo que requiere cada iterable antes de arrancar está en la lámina 8.

4 Tiempos y cambios

- Las 11 semanas cuentan desde que está listo lo que requieren el proyecto y el iterable 1 (lámina 8).
- Si lo que requiere un iterable no está listo antes de que arranque, ese iterable y la fecha final se corren el mismo tiempo.
- Cada iterable se libera cuando Viva lo acepta; la conexión con la PUI con las tres fases es, como pronto, en la semana 12 y depende de Viva y la PUI.
- Los cambios posteriores al inicio se estiman aparte.

5 Aceptación

- **Viva acepta cada iterable antes de liberarlo** y aprueba la aceptación final con el responsable funcional que designe.
- Las validaciones en QA, Producción o sandbox de la PUI las ejecuta Viva o la PUI, o se acompañan cuando habiliten los accesos.
- Defectos de la solución: se corrigen sin costo durante las 2 semanas de soporte.
- Fallas por cambios en contratos de la API interna o de la PUI: cambio de alcance, se cobra aparte.

6 Inversión

- Valor del proyecto: **[PENDIENTE]**
- Forma de pago, por ejemplo por iterable aceptado: **[PENDIENTE]**
- Continuidad del soporte después de las 2 semanas, si Viva la requiere: **[PENDIENTE]**

Soporte de 2 semanas después de liberar a Producción

1 Qué incluye

- Atención de incidentes y consultas sobre la solución.
- Diagnóstico con los logs, métricas y la bitácora que Viva comparta.
- Corrección de defectos y de las vulnerabilidades que reporte Viva.
- Acompañamiento en los despliegues de corrección que ejecute Viva.

2 Cómo funciona

- Arranca con la liberación a Producción del último iterable y dura 2 semanas (semanas 12 y 13).
- Está incluido en el valor del proyecto.
- Cierra con un informe de lo atendido.
- Horario laboral normal, con respuesta en las siguientes 24 horas.

3 Qué no incluye

- Operación 24×7 ni guardias.
- Acceso o despliegues directos en QA y Producción: los ejecuta Viva.
- Funcionalidades nuevas: se estiman aparte.
- Adaptaciones por cambios en los contratos de la API interna o de la PUI: son cambios de alcance y se cobran aparte.

Después de las 2 semanas: si Viva requiere continuidad, el soporte se acuerda y se cotiza aparte del proyecto.